SENAI

Front-End Interativo

APOSTILA DO TIO ALCEU

> Módulo: Dominando o DOM com JS

O Guia para resolver os 20 Desafios de Lógica e Interface

HTML CSS JS

Sumário Interativo

01. Selecionando Elementos (Pegando o HTML pelo braço) 01
02. Ouvindo Eventos (Cliques, Teclado, Mouse) 02
03. Manipulando Classes e Estilos (A Mágica Visual) 03
04. Conteúdo, Inputs e Atributos (Lendo dados do usuário) 04
05. Controlando o Tempo (Temporizadores) 05

Fala, dev! Para resolver os 20 desafios (como o Dark Mode, Carrinho de Compras, Lâmpada Mágica, etc.), você não precisa decorar milhares de comandos. Você só precisa entender **como o JavaScript conversa com o HTML**. Isso se chama Manipulação do DOM (Document Object Model).

01. Selecionando Elementos

Antes de dar uma ordem a um elemento na tela (como "mude de cor" ou "esconda-se"), o JavaScript precisa **encontrar** esse elemento. Usamos os "Seletores".

A Ferramenta de Ouro: document.querySelector

Funciona exatamente como os seletores do CSS. Pega o **primeiro** elemento que encontrar.

index.html script.js

```
/* Exemplo de Estrutura HTML */
<button id="meu-botao" class="btn-primario"> Clique Aqui </button>

/* Exemplo de Aplicação (JavaScript) */
const botao = document.querySelector('#meu-botao');
const botaoPelaClasse = document.querySelector('.btn-primario');
```

Exemplo de Resultado Esperado:

O JS cria uma variável chamada `botao` que agora "segura" o botão do HTML na memória, pronta para receber comandos.

02. Ouvindo Eventos

O JavaScript fica "escutando" as ações do usuário (clique, passar o mouse, digitar). Quando a ação acontece, ele dispara uma função. Usamos o `addEventListener`.

Dica do Tio Alceu

Os eventos mais usados nos desafios são: `'click'` (clicar), `'mouseover'` (passar o mouse em cima), `'mouseout'` (tirar o mouse de cima) e `'input'` (quando o usuário digita algo em um campo).

script.js

```
/* Exemplo de Aplicação: Botão de Votação (Coração) */
const coracao = document.querySelector('.icone-coracao');

coracao.addEventListener('click', () => {
  console.log("O usuário clicou no coração!");
});
```

03. Manipulando Classes e Estilos

A melhor forma de alterar o visual (como no **Dark Mode** ou na **Lâmpada Mágica**) não é mexendo no estilo direto via JS, mas sim adicionando ou removendo classes CSS pré-criadas. Usamos o `classList`.

`.classList.add()`

Adiciona uma classe ao elemento.

`.classList.remove()`

Remove uma classe do elemento.

```
lampada.classList.add('acesa');
```

```
lampada.classList.remove('acesa');
```

🪄 .classList.toggle() (O Melhor Amigo)

Se a classe existir, ele remove. Se não existir, ele adicioniona. Perfeito para botões de "Ligar/Desligar", "Curtir/Descurtir", etc.

```
botaoDarkMode.addEventListener('click', () => {
  document.body.classList.toggle('modo-escuro');
});
```

Exemplo de Resultado (Desafio Dark Mode):

Ao clicar no botão, o `` do seu HTML ganha a classe `modo-escuro`. Como você já definiu no CSS que `.modo-escuro { background: black; color: white; }`, a tela inverte as cores automaticamente!

04. Conteúdo, Inputs e Atributos

Muitos desafios exigem ler o que o usuário digitou (como na **Calculadora de Gorjeta** ou **Aprovador de Cores**), ou trocar a imagem (como na **Galeria de Arte**).

PROPRIEDADE	PARA QUE SERVE?	EXEMPLO PRÁTICO
`.value`	Pega o que foi digitado em um <input> ou <textarea>.	`let senha = inputSenha.value;`
`.innerText`	Altera o texto visível dentro de uma Tag (ex: <h1>, <p>).	`titulo.innerText = "Novo Placar!";`
`.src`	Muda o link da imagem (usado na Lâmpada e Galeria).	`img.src = 'lamp_acesa.jpg';`
`.disabled`	Desativa um botão (true/false) (Usado no Contador do Twitter).	`botao.disabled = true;`

Aplicação: Revelador de Senha Estiloso

```
/* Lógica: Quando apertar o mouse no ícone, muda o tipo do input de
"password" para "text". Ao soltar, volta. */

iconeOlho.addEventListener('mousedown', () => {
  meuInput.type = 'text';
});

iconeOlho.addEventListener('mouseup', () => {
  meuInput.type = 'password';
});
```

⏱ 05. Controlando o Tempo

Desafios como o **Cronômetro Pomodoro** e o **Jogo da Memória** (para virar a carta de volta após um tempo) dependem de funções nativas de temporização do JavaScript.

1. setTimeout (Atraso)

Executa uma função **uma única vez**, após esperar um tempo (em milissegundos).

```
// Espera 1 segundo (1000ms) para
virar a carta
setTimeout(() => {
  carta.classList.remove('virada');
}, 1000);
```

2. setInterval (Repetição)

Executa uma função **repetidamente**, a cada X milissegundos. Útil para cronômetros.

```
// Atualiza o relógio a cada 1
segundo
const relogio = setInterval(() =>
{
  segundos_restantes--;
  atualizaTela();
}, 1000);
```

Importante: Para parar um `setInterval` (como quando o cronômetro chega a zero ou o usuário pausa), você usa a função `clearInterval(relogio)` passando a variável do intervalo.

Você está pronto!

Pegue um café, abra seu VS Code e comece pelo Desafio 1. Boa sorte!